

RISC-V Processor - Phase 1

Team Name: RISK_3

Team Members:

1. Nikhil Venkat Atkuru – Roll No: 2024102039
2. Nagamalla Sai Abhinav – Roll No: 2024102037
3. Abhinav Venkata Kota – Roll No: 2024102059

1 Introduction

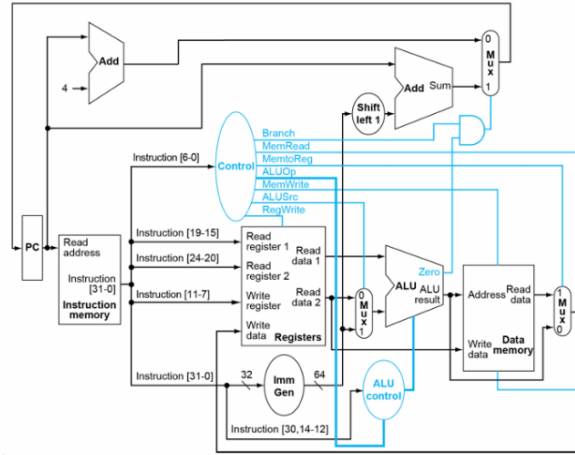


Figure 1: Sequential datapath. From Ch.4, Computer Organization and Design by D. Patterson and J. Hennessey

This phase of the project is the implementation of a single-cycle, non-pipelined RISC-V processor datapath supporting a limited subset of the RISC-V ISA. In a single-cycle architecture, each instruction completes all stages of execution within one clock cycle, simplifying control and datapath design at the cost of reduced performance. While the single-cycle approach offers conceptual clarity and ease of verification, it suffers from limitations such as long critical paths and inefficient hardware utilization. These drawbacks motivate the introduction of pipelining, which is the next phase of the project. The course reference textbook was used for the design and implementation which provided a clear explanation of the required datapath components and control logic Sequential datapath. From Ch.4, Computer Organization and Design by D. Patterson and J. Hennessey

2 ALU

The ALU is the primary execution block of the processor. It performs arithmetic and logical operations on the two 64-bit input operands based on the control signal generated by the ALU Control block. It takes data from the registers, processes it accordingly, and outputs the result to be stored or for memory addressing. A few modifications are made to the one submitted for the assignment to just use one adder for the whole

ALU instead of 4 (one each for add, sub, slt and sltu). The unnecessary flags are removed from the original implementation and now it just outputs a zero flag which is useful in implementing beq (break when equal) branching operation. Also, the zero flag is now checked only once instead of checking in every module. These changes will reduce the amount of hardware, saving space on the chip.

```
sai@sai-Victus-by-HP-Gaming-Laptop-15-falxxx:~/Desktop/Projects/RISC-V-Processor/ALU$ iverilog alu_tb.v
sai@sai-Victus-by-HP-Gaming-Laptop-15-falxxx:~/Desktop/Projects/RISC-V-Processor/ALU$ vvp a.out
----- ALU 64-BIT TEST START -----
PASS | op=0 a=0000000000000000 b=0000000000000000 -> 0000000000000000
PASS | op=0 a=0000000000000001 b=0000000000000001 -> 0000000000000002
PASS | op=0 a=ffffffffffffffff b=0000000000000001 -> 0000000000000000
PASS | op=0 a=7fffffffffffffff b=0000000000000001 -> 8000000000000000
ADD Done

PASS | op=8 a=0000000000000005 b=0000000000000003 -> 0000000000000002
PASS | op=8 a=0000000000000003 b=0000000000000005 -> ffffffffffffffff
PASS | op=8 a=0000000000000000 b=0000000000000000 -> 0000000000000000
SUB Done

PASS | op=2 a=7fffffffffffffff b=fffffffffffffff -> 0000000000000000
PASS | op=2 a=fffffffffffffff b=7fffffffffffffff -> 0000000000000001
PASS | op=2 a=0000000000000005 b=0000000000000006 -> 0000000000000001
PASS | op=2 a=0000000000000006 b=0000000000000005 -> 0000000000000000
PASS | op=2 a=fffffffffffffff b=fffffffffffffff1 -> 0000000000000001
SLT Done

PASS | op=3 a=0000000000000005 b=0000000000000006 -> 0000000000000001
PASS | op=3 a=0000000000000006 b=0000000000000005 -> 0000000000000000
PASS | op=3 a=fffffffffffffff b=0000000000000000 -> 0000000000000000
SLTU Done

PASS | op=4 a=0000000000000000 b=0000000000000000 -> 0000000000000000
PASS | op=4 a=fffffffffffffff b=0000000000000000 -> ffffffffffffffff
PASS | op=4 a=aaaaaaaaaaaaaaaa b=5555555555555555 -> ffffffffffffffff
XOR Done

PASS | op=6 a=0000000000000000 b=0000000000000000 -> 0000000000000000
PASS | op=6 a=0000000000001234 b=000000000000abcd -> 000000000000bbfd
OR Done

PASS | op=7 a=fffffffffffffff b=0000000000000000 -> 0000000000000000
PASS | op=7 a=f0f0f0f0f0f0f0f0 b=0f0f0f0f0f0f0f0f -> 0000000000000000
AND Done

PASS | op=1 a=0000000000000001 b=0000000000000001 -> 0000000000000002
PASS | op=1 a=0000000000000001 b=0000000000000003 -> 0000000000000000
SLL Done

PASS | op=5 a=8000000000000000 b=0000000000000001 -> 4000000000000000
SRL Done

PASS | op=d a=8000000000000000 b=0000000000000001 -> c000000000000000
SRA Done

PASS | op=f a=0000000000001234 b=0000000000005678 -> 0000000000000000
Invalid 4 bit control signal Done
```

Figure 2: ALU testbench result (1).

```
-----
TOTAL TESTS : 27
PASSED      : 27
FAILED      : 0
-----
ALL TESTS PASSED
alu_tb.v:116: $finish called at 27000 (1ps)
```

Figure 3: ALU testbench result (2).

3 Control

This module takes the opcode as input and generates the necessary control signals for the rest of the processor for various operations like reading, loading, branching etc. Each of them are described in detail below:

```
sai@sai-Victus-by-HP-Gaming-Laptop-15-falxxx:~/Desktop/Projects/RISC-V-Processor$ iverilog control_tb.v
sai@sai-Victus-by-HP-Gaming-Laptop-15-falxxx:~/Desktop/Projects/RISC-V-Processor$ vvp a.out
---- CONTROL UNIT TEST ----
R format instruction
PASS: opcode=0110011
I format instruction
PASS: opcode=0010011
Load instruction
PASS: opcode=0000011
Store instruction
PASS: opcode=0100011
Branch instruction
PASS: opcode=1100011
-----
TOTAL TESTS : 5
PASSED      : 5
FAILED      : 0
-----
ALL TESTS PASSED
control_tb.v:92: $finish called at 5000 (1ps)
sai@sai-Victus-by-HP-Gaming-Laptop-15-falxxx:~/Desktop/Projects/RISC-V-Processor$
```

Figure 4: Control module testbench result.

Input or output	Signal name	R-format	ld	sd	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Figure 5: Control module diagram.

- **Branch:** This is asserted high for a branch instruction. Since we are implementing only **beq**, it will branch if both branch and zero_flag are high, which is checked by a 2 input AND gate.
- **MemRead:** This is asserted high if we want to read from the data memory i.e. for load instruction.

- **MemtoReg:** This is asserted high to write the data from the data memory to the register file i.e. for load instruction in the write back stage. If it is 0, then the data which should be written back to the register is taken from the ALU output. It acts as a control to the MUX which decides what data is to be written back.
- **MemWrite:** This is asserted high if we want to write into the data memory i.e. for store instruction. The data which is to be written will come from the register file and will be written to the address calculated by the ALU after adding the base address and the offset.
- **ALUSrc:** It is a control signal to one of the MUXs. It is asserted high for immediate, load and store format instructions where the second operand of the ALU is the result of the immediate generate block. When it's low, then the second operand is from the register file. For branch operation, it is asserted low because both the operands are coming from the register file for checking for equality (which is done by subtracting both the operands and checking for zero flag).
- **RegWrite:** It is asserted high for writing into the register file. So, for operations like R type, I type and load type, the data which is to be written in the write back stage will be written into the register selected by the `rd` of the instruction.
- **ALUOp:** It is a 2 bit signal which tells the ALUControl block to select one of the many operations of the ALU. How each operation is selected is explained under ALUControl block but for now, we need to know that
 - $ALUOp = 00$ for load/store type instructions
 - $ALUOp = 01$ for branching instruction
 - $ALUOp = 10$ for R format instruction
 - $ALUOp = 11$ for I format instruction

4 MUXs

There are 3 2*1 MUXes, all of them are 64 bit wide which have the following uses:

- **PC MUX:** It is used to determine if the next instruction is the immediate instruction ($PC + 4$) or the instruction + offset (due to branching). So, its select line is the AND of branch and zero_flag.
- **ALU Operand MUX:** It is used to determine whether the second operand of the ALU is from the immediate generate block or from the register file. It has a select line coming directly from the control block (ALUSrc).
- **Write back MUX:** It selects which data should be written to the register file in the write back stage, controlled by the control signal MemtoReg. If the control signal is 1, the data from the memory is written into the register file, else the data from the ALU output is written back.

5 PC Adder

The PC adder is a 64-bit adder used to compute the next instruction address. Its inputs are the current PC value and an offset selected by the PC multiplexer, which is either the constant value 4 or the branch offset. In the reference design, two separate adders are used to compute $PC + 4$ and $PC + branch_offset$, and a multiplexer selects between their outputs. In the modified design, the multiplexer is moved to select the second operand of a single adder instead of selecting between two adder outputs. This produces the same result while eliminating one 64-bit adder, thereby reducing hardware area.

6 Shift Left by 1

The shift-left-by-1 block is used to align the branch offset before computing the branch target address. In RISC-V, branch targets are at least halfword aligned, which means the least significant bit of the branch offset is always zero. Shifting the immediate left by one bit enforces this alignment. Although shifting left by two bits is sufficient for a design that supports only 32-bit instructions, using a shift left by one bit makes the design compatible with the general RISC-V ISA, including systems that support compressed (16-bit) instructions. This ensures correct byte alignment while maintaining broader architectural compatibility.

7 PC

The Program Counter (PC) is a 64-bit register that stores the memory address of the next instruction to be fetched. In the RISC-V architecture, each instruction is 32 bits (4 bytes), so the PC normally increments by 4 after every instruction since memory is byte-addressed. In the case of branch instructions, the PC is updated with the computed target address using the given offset. The PC is updated on every clock edge in a sequential design, where one instruction is completed per clock cycle in a single-cycle implementation. The reset is implemented synchronously.

```
Test Case 0:
Test: pc_in = deadbeefdeadbeef
reset = 1
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

Test Case 1:
Test: pc_in = 0000000000000004
reset = 0
Expected pc_out = 0000000000000004
Actual pc_out = 0000000000000004
Status: PASS

Test Case 2:
Test: pc_in = 0000000000000000
reset = 0
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

Test Case 3:
Test: pc_in = ffffffffcccccccc
reset = 0
Expected pc_out = ffffffffcccccccc
Actual pc_out = ffffffffcccccccc
Status: PASS

Test Case 4:
Test: pc_in = 0000000000001000
reset = 1
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

Test Case 5:
Test: pc_in = 0000000000000008
reset = 0
Expected pc_out = 0000000000000008
Actual pc_out = 0000000000000008
Status: PASS

Test Case 6:
Test: pc_in = 000000000000000c
reset = 0
Expected pc_out = 000000000000000c
Actual pc_out = 000000000000000c
Status: PASS

Test Case 7:
Test: pc_in = 0000000000002000
reset = 0
Expected pc_out = 0000000000002000
Actual pc_out = 0000000000002000
Status: PASS

Test Case 8:
Test: pc_in = 0000000000002004
reset = 1
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

=====
Test completed
=====
ALL TESTS PASSED
```

Figure 6: PC block.

```
Test Case 0:
Test: pc_in = deadbeefdeadbeef
reset = 1
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

Test Case 1:
Test: pc_in = 0000000000000004
reset = 0
Expected pc_out = 0000000000000004
Actual pc_out = 0000000000000004
Status: PASS

Test Case 2:
Test: pc_in = 0000000000000000
reset = 0
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

Test Case 3:
Test: pc_in = ffffffffcccccccc
reset = 0
Expected pc_out = ffffffffcccccccc
Actual pc_out = ffffffffcccccccc
Status: PASS

Test Case 4:
Test: pc_in = 0000000000001000
reset = 1
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

Test Case 5:
Test: pc_in = 0000000000000008
reset = 0
Expected pc_out = 0000000000000008
Actual pc_out = 0000000000000008
Status: PASS

Test Case 6:
Test: pc_in = 000000000000000c
reset = 0
Expected pc_out = 000000000000000c
Actual pc_out = 000000000000000c
Status: PASS

Test Case 7:
Test: pc_in = 0000000000002000
reset = 0
Expected pc_out = 0000000000002000
Actual pc_out = 0000000000002000
Status: PASS

Test Case 8:
Test: pc_in = 0000000000002004
reset = 1
Expected pc_out = 0000000000000000
Actual pc_out = 0000000000000000
Status: PASS

=====
Test completed
=====
ALL TESTS PASSED
```

Figure 7: PC block.

8 ImmGen

The Immediate Generator block takes the 32-bit instruction as input. Based on the opcode, it determines the instruction format (I-type, S-type, or B-type) and extracts the corresponding 12-bit immediate field from different bit positions of the instruction. The extracted immediate is then sign-extended to 64 bits so that it can be used correctly in 64-bit arithmetic operations.

```

=====
Immediate Generator Test Bench
=====

Test 1: I-type immediate (addi)
Instruction: 0x00518193
Opcode: 0b0010011, Funct3: 0b000
Immediate: 0x0000000000000005 (5)
Expected: 0x0000000000000005 (5)
Status: PASS

Test 2: I-type with negative immediate
Instruction: 0xffb18193
Opcode: 0b0010011, Funct3: 0b000
Immediate: 0xfffffffffffffffb (-5)
Expected: 0xfffffffffffffffb (-5)
Status: PASS

Test 3: I-type load (lw)
Instruction: 0x0041a103
Opcode: 0b0000011, Funct3: 0b010
Immediate: 0x0000000000000004 (4)
Expected: 0x0000000000000004 (4)
Status: PASS

Test 4: S-type store (sw)
Instruction: 0x0021a223
Opcode: 0b0100011, Funct3: 0b010
Immediate: 0x0000000000000004 (4)
Expected: 0x0000000000000004 (4)
Status: PASS

Test 5: S-type with larger immediate
Instruction: 0x0641a223
Opcode: 0b0100011, Funct3: 0b010
Immediate: 0x0000000000000064 (100)
Expected: 0x0000000000000064 (100)
Status: PASS

```

Figure 8: Immediate generator (1).

```

Test 6: B-type branch (beq)
Instruction: 0x00208263
Opcode: 0b1100011, Funct3: 0b000
Immediate: 0x0000000000000002 (2)
Expected: 0x0000000000000002 (2)
Status: PASS

Test 7: B-type with negative offset
Instruction: 0xfe208ce3
Opcode: 0b1100011, Funct3: 0b000
Immediate: 0xfffffffffffffc (-4)
Expected: 0xfffffffffffffc (-4)
Status: PASS

Test 8: I-type shift immediate (slli)
Instruction: 0x00519193
Opcode: 0b0010011, Funct3: 0b001
Immediate: 0x0000000000000005 (5)
Expected: 0x0000000000000005 (5)
Status: PASS

Test 9: I-type shift immediate (srai)
Instruction: 0x40a1d193
Opcode: 0b0010011, Funct3: 0b101
Immediate: 0x000000000000000a (10)
Expected: 0x000000000000000a (10)
Status: PASS

Test 10: Load with negative offset
Instruction: 0xffc1a103
Opcode: 0b0000011, Funct3: 0b010
Immediate: 0xfffffffffffffc (-4)
Expected: 0xfffffffffffffc (-4)
Status: PASS

Test 11: Unknown opcode
Instruction: 0x00000000
Opcode: 0b0000000, Funct3: 0b000
Immediate: 0x0000000000000000 (0)
Expected: 0x0000000000000000 (0)
Status: PASS

Test 12: Store with negative offset
Instruction: 0xfe21aa23
Opcode: 0b0100011, Funct3: 0b010
Immediate: 0xfffffffffffff4 (-12)
Expected: 0xfffffffffffff4 (-12)
Status: PASS

=====
Test completed!
=====
ALL TESTS PASSED

```

Figure 9: Immediate generator (2).

```

=====
Immediate Generator Test Bench
=====

Test 1: I-type immediate (addi)
Instruction: 0x00518193
Opcode: 0b0010011, Funct3: 0b000
Immediate: 0x0000000000000005 (5)
Expected: 0x0000000000000005 (5)
Status: PASS

Test 2: I-type with negative immediate
Instruction: 0xffb18193
Opcode: 0b0010011, Funct3: 0b000
Immediate: 0xfffffffffffffffb (-5)
Expected: 0xfffffffffffffffb (-5)
Status: PASS

Test 3: I-type load (lw)
Instruction: 0x0041a103
Opcode: 0b0000011, Funct3: 0b010
Immediate: 0x0000000000000004 (4)
Expected: 0x0000000000000004 (4)
Status: PASS

Test 4: S-type store (sw)
Instruction: 0x0021a223
Opcode: 0b0100011, Funct3: 0b010
Immediate: 0x0000000000000004 (4)
Expected: 0x0000000000000004 (4)
Status: PASS

Test 5: S-type with larger immediate
Instruction: 0x0641a223
Opcode: 0b0100011, Funct3: 0b010
Immediate: 0x0000000000000064 (100)
Expected: 0x0000000000000064 (100)
Status: PASS

```

Figure 10: Immgen 1 (alt).

```

Test 6: B-type branch (beq)
Instruction: 0x00208263
Opcode: 0b1100011, Funct3: 0b000
Immediate: 0x0000000000000002 (2)
Expected: 0x0000000000000002 (2)
Status: PASS

Test 7: B-type with negative offset
Instruction: 0xfe208ce3
Opcode: 0b1100011, Funct3: 0b000
Immediate: 0xfffffffffffffc (-4)
Expected: 0xfffffffffffffc (-4)
Status: PASS

Test 8: I-type shift immediate (slli)
Instruction: 0x00519193
Opcode: 0b0010011, Funct3: 0b001
Immediate: 0x0000000000000005 (5)
Expected: 0x0000000000000005 (5)
Status: PASS

Test 9: I-type shift immediate (srai)
Instruction: 0x40a1d193
Opcode: 0b0010011, Funct3: 0b101
Immediate: 0x000000000000000a (10)
Expected: 0x000000000000000a (10)
Status: PASS

Test 10: Load with negative offset
Instruction: 0xffc1a103
Opcode: 0b0000011, Funct3: 0b010
Immediate: 0xfffffffffffffc (-4)
Expected: 0xfffffffffffffc (-4)
Status: PASS

Test 11: Unknown opcode
Instruction: 0x00000000
Opcode: 0b0000000, Funct3: 0b000
Immediate: 0x0000000000000000 (0)
Expected: 0x0000000000000000 (0)
Status: PASS

Test 12: Store with negative offset
Instruction: 0xfe21a223
Opcode: 0b0100011, Funct3: 0b010
Immediate: 0xfffffffffffffc (-12)
Expected: 0xfffffffffffffc (-12)
Status: PASS

=====
Test completed!
=====
ALL TESTS PASSED

```

Figure 11: Immgen 2 (alt).

9 Register File

The Register File is a storage unit containing 32 general-purpose registers (x0 to x31), each 32 bits wide. Register x0 is hardwired to zero and always returns 0 regardless of what is written to it. The register file has two read ports and one write port, enabling simultaneous reading of two source registers and writing to one destination register. The inputs to this block are `clk`, `reset`, `read_reg1` (5-bit address for first source register), `read_reg2` (5-bit address for second source register), `write_reg` (5-bit address for destination register), `write_data` (32-bit data to be written), and `reg_write` (control signal to enable write operation). The outputs are `read_data1` and `read_data2` (both 32-bit), which provide the data stored in the registers specified by `read_reg1` and `read_reg2` respectively. Read operations are asynchronous, meaning the register data appears at the output immediately when the address is provided. Write operations occur on the rising edge of the clock when `reg_write` is enabled. The register file stores temporary data, operands for the ALU, and results from operations during program execution.

```

=====
Register File Test Bench
=====

Test 1: Reset
After reset: reg1=1, output1_r=0 (expected 0)
After reset: reg2=2, output2_r=0 (expected 0)

Test 2: Write and Read
Write 0xdeadbeefcafebabe to reg5, Read reg5: 0xdeadbeefcafebabe

Test 3: Multiple registers
Read reg5: 0xdeadbeefcafebabe (expected 0xDEADBEEFCAFEBA)
Read reg10: 0x123456789abcdef0 (expected 0x123456789ABCDEF0)

Test 4: x0 hardwired to 0
Write to x0, Read x0: 0x0000000000000000 (expected 0x0000000000000000)

Test 5: Read old data during write
After write to reg15: 0xaaaaaaaa55555555 (expected 0xAAAAAAAA55555555)

Test 6: Testing multiple registers
Test completed!

=====
register_file_tb.v:99: $finish called at 100000 (1ps)

```

Figure 12: Register file.

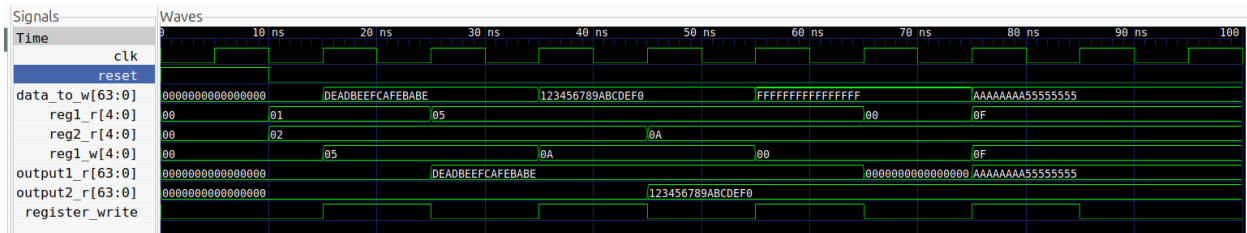


Figure 13: Register file waveform.

10 Data Memory

The Data Memory is a memory block used for storing data that the program operates on during execution. The defined size of the data memory is 4096 bytes. Unlike the instruction memory, the data memory supports both read and write operations. The inputs to this block are `clk`, `reset`, `addr` (32-bit address), `write_data` (32-bit data to be stored), `mem_read` (control signal to enable read operation), and `mem_write` (control signal to enable write operation). The output is `read_data` (32-bit), which provides the data read from the specified memory location. Each memory location stores 1 byte, and the memory is byte-addressable. For read operations, when `mem_read` is asserted, the memory accesses four consecutive bytes starting from the given address, assembles them in little-endian format, and produces the 32-bit output. For write operations, when `mem_write` is asserted on the rising edge of the clock, the 32-bit `write_data` is split into four bytes and stored in four consecutive memory locations starting from the specified address. This memory is primarily used by load and store instructions to transfer data between registers and memory during program execution.


```

=====
Data Memory Test Bench
Big-endian byte ordering
=====

Test 1: Reset
After reset, read addr 0: 0x0000000000000000 (expected 0x0000000000000000)

Test 2: Write and Read (Big-endian)
Write 0xdeadbeefcafebabe to addr 0
Read from addr 0: 0xdeadbeefcafebabe
Big-endian storage: mem[0]=0xde, mem[1]=0xad, mem[2]=0xbe, mem[3]=0xef
Big-endian storage: mem[4]=0xca, mem[5]=0xfe, mem[6]=0xba, mem[7]=0xbe

Test 3: Write to address 8
Write 0x123456789abcdef0 to addr 8
Read from addr 8: 0x123456789abcdef0

Test 4: Verify previous data intact
Read from addr 0: 0xdeadbeefcafebabe (expected 0xDEADBEEFCAFEBA)

Test 5: Write disabled test
Write disabled, read addr 0: 0xdeadbeefcafebabe (expected 0xDEADBEEFCAFEBA)

Test 6: Read disabled test
mem_r=0, output_w: 0x0000000000000000 (expected 0x0000000000000000)

Test 7: Unaligned access (addr=4)
Write 0xaabbccddeeff0011 to addr 4
Read from addr 4: 0xcafebabe12345678

=====
Test completed!
=====
data_memory_tb.v:121: $finish called at 110000 (1ps)

```

Figure 14: Data memory.

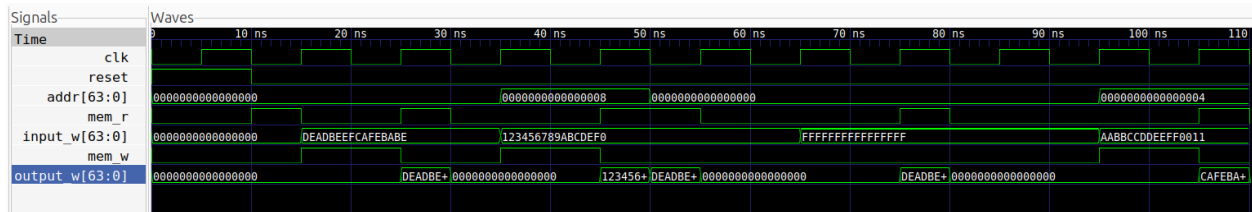


Figure 15: Data memory waveform.

11 Instruction Memory

The Instruction Memory is a memory block which allows only the read operation. The defined size of the instruction memory block is 4096 bytes. This instruction memory stores the instructions which are to be executed. The inputs to this block are clk, reset, and addr, and the output provided by this block is a 32-bit instruction. Each memory location stores 1 byte. Instructions are stored in another file named `instructions.txt`, where each line corresponds to 1 byte, and thus every 4 lines (which is 4 bytes – 32 bits) correspond to one instruction. The instruction memory reads this file and the memory is filled accordingly. The way the memory works is that when the address is received as input, it accesses four consecutive memory locations starting from the given address and concatenates them to produce the 32-bit instruction.

```

=====
Instruction Memory Test Bench
=====

Test 1: Read instruction at address 0
Address: 0x000
Instruction: 0x00500113 (expected: 0x00500113 = addi x2, x0, 5)

Test 2: Read instruction at address 4
Address: 0x004
Instruction: 0x00a00193 (expected: 0x00A00193 = addi x3, x0, 10)

Test 3: Read instruction at address 8
Address: 0x008
Instruction: 0x003102b3 (expected: 0x003102B3 = add x5, x2, x3)

Test 4: Read instruction at address 12
Address: 0x00c
Instruction: 0x0020f233 (expected: 0x0020F233 = and x4, x1, x2)

Test 5: Sequential reads
Addr 0x000: 0x00500113
Addr 0x004: 0x00a00193
Addr 0x008: 0x003102b3
Addr 0x00c: 0x0020f233

Test 6: Address beyond loaded data
Address: 0x100
Instruction: 0x00000000 (expected: 0x00000000)

=====
Test completed!
=====

```

Figure 16: Instruction memory.

12 ALU Control

The ALU Control block is responsible for generating the specific control signal which is required to select the operation which needs to be executed by the ALU. The ALUop signal is provided by the main control block which specifies the type of instruction whether it is load/store, branch, R-type or I-type instructions. But to determine the exact ALU function alongside the ALUop signal we also need the funct3 and funct7 fields from the instruction to identify the exact operation. Based on these 3 inputs the ALU control block produces the 4 bit output control signal to the ALU which controls the internal operations of the ALU. Thus it acts as an intermediate block between the main control unit and the ALU ensuring the correct operation is being executed for the given instruction.

```

=====
ALU Control Unit Test Bench
=====
ALUOp encoding:
00 = Load/Store (ADD)
01 = Branch (SUB)
10 = R-type (use funct7/funct3)
11 = I-type (use funct3)

--- ALUOp = 00 (Load/Store) ---
Test:
ALUOp=00, Instr[3:0]=0b0000 (funct7_6=0, funct3=000)
ALU control: 0b0000 (expected 0b0000)
Status: PASS

Test:
ALUOp=00, Instr[3:0]=0b1111 (funct7_6=1, funct3=111)
ALU control: 0b0000 (expected 0b0000)
Status: PASS

--- ALUOp = 01 (Branch) ---
Test:
ALUOp=01, Instr[3:0]=0b0000 (funct7_6=0, funct3=000)
ALU control: 0b1000 (expected 0b1000)
Status: PASS

Test:
ALUOp=01, Instr[3:0]=0b1111 (funct7_6=1, funct3=111)
ALU control: 0b1000 (expected 0b1000)
Status: PASS

--- ALUOp = 10 (R-type) ---
Test:
ALUOp=10, Instr[3:0]=0b0000 (funct7_6=0, funct3=000)
ALU control: 0b0000 (expected 0b0000)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b1000 (funct7_6=1, funct3=000)
ALU control: 0b1000 (expected 0b1000)
Status: PASS

```

Figure 17: ALU control (1).

```

Test:
ALUOp=10, Instr[3:0]=0b0111 (funct7_6=0, funct3=111)
ALU control: 0b0111 (expected 0b0111)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b0110 (funct7_6=0, funct3=110)
ALU control: 0b0110 (expected 0b0110)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b0100 (funct7_6=0, funct3=100)
ALU control: 0b0100 (expected 0b0100)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b0001 (funct7_6=0, funct3=001)
ALU control: 0b0001 (expected 0b0001)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b0101 (funct7_6=0, funct3=101)
ALU control: 0b0101 (expected 0b0101)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b1101 (funct7_6=1, funct3=101)
ALU control: 0b1101 (expected 0b1101)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b0010 (funct7_6=0, funct3=010)
ALU control: 0b0010 (expected 0b0010)
Status: PASS

Test:
ALUOp=10, Instr[3:0]=0b0011 (funct7_6=0, funct3=011)
ALU control: 0b0011 (expected 0b0011)
Status: PASS

```

Figure 18: ALU control (2).

```

--- ALUOp = 11 (I-type immediate) ---
Test:
ALUOp=11, Instr[3:0]=0b0000 (funct7_6=0, funct3=000)
ALU control: 0b0000 (expected 0b0000)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b0111 (funct7_6=0, funct3=111)
ALU control: 0b0111 (expected 0b0111)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b0110 (funct7_6=0, funct3=110)
ALU control: 0b0110 (expected 0b0110)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b0100 (funct7_6=0, funct3=100)
ALU control: 0b0100 (expected 0b0100)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b0010 (funct7_6=0, funct3=010)
ALU control: 0b0010 (expected 0b0010)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b0011 (funct7_6=0, funct3=011)
ALU control: 0b0011 (expected 0b0011)
Status: PASS

```

Figure 19: ALU control (3).

```

Test:
ALUOp=11, Instr[3:0]=0b0001 (funct7_6=0, funct3=001)
ALU control: 0b0001 (expected 0b0001)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b0101 (funct7_6=0, funct3=101)
ALU control: 0b0101 (expected 0b0101)
Status: PASS

Test:
ALUOp=11, Instr[3:0]=0b1101 (funct7_6=1, funct3=101)
ALU control: 0b1101 (expected 0b1101)
Status: PASS

--- Default case ---
Test:
ALUOp=xx, Instr[3:0]=0b0000 (funct7_6=0, funct3=000)
ALU control: 0b0000 (expected 0b0000)
Status: PASS

=====
Test completed
=====
ALL TESTS PASSED

```

Figure 20: ALU control (4).

13 Tested Codes

13.1 Code 1

The first program tested was the given sample test case, which has all the required ALU operations specified in the assignment. The observed register outputs match the expected results, confirming the correctness of the implemented instructions and control logic.

```

----- Register File Contents -----
x0 = 0
x1 = 15
x2 = -5
x3 = 10
x4 = 10
x5 = 10
x6 = -5
x7 = -5
x8 = 0
x9 = 0
x10 = 15
x11 = -5
x12 = 0
x13 = 30
x14 = 0
x15 = 0
x16 = 0
x17 = 0
x18 = 0
x19 = 0
x20 = 0
x21 = 0
x22 = 0
x23 = 0
x24 = 0
x25 = 0
x26 = 0
x27 = 0
x28 = 0
x29 = 0
x30 = 0
x31 = 0
-----
Total cycles = 15

```

Figure 21: Result for the given instruction .txt

13.2 Code 2

The second test program computes the first ten Fibonacci numbers using a single loop. The register values obtained at the end of execution correspond to the correct Fibonacci sequence, verifying the load, store, branching operations.

```

addi x2, x0, 5      # x2 = 5
addi x3, x0, 10     # x3 = 10
add x1, x2, x3      # x1 = x2 + x3 = 15
sub x2, x2, x3      # x2 = x2 - x3 = -5
and x4, x3, x3      # x4 = x3 & x3 = 10
and x5, x3, x4      # x5 = x3 & x4 = 10
or x6, x2, x4       # x6 = x2 | x4 = -5
or x7, x2, x3       # x7 = x2 | x3 = -5
sd x1, 0(x5)        # x1 is stored at memory address x5 + 0 (10 + 0 = 10)
ld x10, 0(x5)        # x10 = memory[x5 + 0] = memory[10] = 15
sd x6, 24(x5)        # x6 is stored at memory address x5 + 24 (10 + 24 = 34)
ld x11, 24(x5)       # x11 = memory[x5 + 24] = memory[34] = -5
beq x4, x5, 8        # if x4 == x5, branch to PC + 8 (so skip the next instruction)
beq x0, x0, 0        # if x0 == x0, branch to PC + 0 (infinite loop)
add x13, x1, x10     # x13 = x1 + x10 = 30

```

Figure 22: Given Assembly code

```

----- Register File Contents -----
x0 = 0
x1 = 10
x2 = 0
x3 = 21
x4 = 34
x5 = 10
x6 = 34
x7 = 0
x8 = 0
x9 = 0
x10 = 0
x11 = 1
x12 = 1
x13 = 2
x14 = 3
x15 = 5
x16 = 8
x17 = 13
x18 = 21
x19 = 34
x20 = 0
x21 = 0
x22 = 0
x23 = 0
x24 = 0
x25 = 0
x26 = 0
x27 = 0
x28 = 0
x29 = 0
x30 = 0
x31 = 0
-----
Total cycles = 86

```

Figure 23: Result for fibonacci series

```

// Fibonacci
// Calculate first n Fibonacci numbers

    addi x1, x0, 10      // n = 10
    addi x2, x0, 0       // base address for storing results (byte address)

    addi x3, x0, 0       // Fib(0) = 0
    addi x4, x0, 1       // Fib(1) = 1

    sd x3, 0(x2)         // store Fib(0)
    addi x2, x2, 8

    sd x4, 0(x2)         // store Fib(1)
    addi x2, x2, 8

    addi x5, x0, 2       // i = 2

fib_loop:
    beq x5, x1, 32       // if i == n, jump to fib_done

    add x6, x3, x4       // next = a + b
    sd x6, 0(x2)         // store next
    addi x2, x2, 8

    addi x5, x5, 1       // i++
    add x3, x0, x4       // a = b
    add x4, x0, x6       // b = next

    beq x0, x0, -28      // jump back to fib_loop

fib_done:
    addi x2, x0, 0       // reset address pointer

    ld x10, 0(x2)        // load Fibonacci numbers
    ld x11, 8(x2)
    ld x12, 16(x2)
    ld x13, 24(x2)
    ld x14, 32(x2)
    ld x15, 40(x2)
    ld x16, 48(x2)
    ld x17, 56(x2)
    ld x18, 64(x2)
    ld x19, 72(x2)

```

Figure 24: Assembly code for fibonacci

13.3 Code 3

The third test program computes the sum of the first thirty natural numbers using a single loop. The final result is stored in register x2, which contains the correct value of 465, validating branching operations.

```

----- Register File Contents -----
x0 = 0
x1 = 30
x2 = 465
x3 = 31
x4 = 1
x5 = 0
x6 = 0
x7 = 0
x8 = 0
x9 = 0
x10 = 1
x11 = 0
x12 = 0
x13 = 0
x14 = 0
x15 = 0
x16 = 0
x17 = 0
x18 = 0
x19 = 0
x20 = 0
x21 = 0
x22 = 0
x23 = 0
x24 = 0
x25 = 0
x26 = 0
x27 = 0
x28 = 0
x29 = 0
x30 = 0
x31 = 0
-----
Total cycles = 154

```

Figure 25: Result for testing sum of natural numbers

13.4 Code 4

Finally, a program containing a double-nested loop executing $5 * 5$ iterations was tested. The program increments a counter by one in each iteration, and the final value stored in register x7 is 25, confirming correct execution of nested loops.

```

addi x1, x0, 30    // n = 30
addi x2, x0, 0     // sum = 0
addi x3, x0, 1     // i = 1 (Loop variable)
add  x10, x3, x0   // x10 = 1 (temp)

Loop:
add  x2, x2, x3    // sum += i
addi x3, x3, 1     // i++
sub  x4, x3, x1    // i - n
beq  x4, x10, 8    // if i == n, branch to Exit
beq  x0, x0, -16   // branch to Loop (unconditional branch)

Exit:

```

Figure 26: Assembly code for sum of natural numbers

```

----- Register File Contents -----
x0 = 0
x1 = 0
x2 = 0
x3 = 0
x4 = 0
x5 = 5
x6 = 5
x7 = 25
x8 = 5
x9 = 0
x10 = 0
x11 = 0
x12 = 0
x13 = 0
x14 = 0
x15 = 0
x16 = 0
x17 = 0
x18 = 0
x19 = 0
x20 = 0
x21 = 0
x22 = 0
x23 = 0
x24 = 0
x25 = 0
x26 = 0
x27 = 0
x28 = 0
x29 = 0
x30 = 0
x31 = 0
-----
Total cycles = 166

```

Figure 27: Result for testing nested condition

```

addi x5, x0, 5
addi x6, x0, 0
addi x7, x0, 0
outer_loop:
sub x10, x6, x5
beq x10, x0, 36
addi x8, x0, 0
inner_loop:
sub x11, x8, x5
beq x11, x0, 16
addi x7, x7, 1
addi x8, x8, 1
beq x0, x0, -16
next_outer:
addi x6, x6, 1
beq x0, x0, -36
done:

```

Figure 28: Assembly code for testing nested condition

14 Additional Features

While we are required to implement a limited subset of R-type instructions, the ALU and control logic are designed following standard RISC-V R and I format conventions, making the design readily extensible to load, store, branch, and jump instructions with minor control-logic modifications.

After analyzing the given table in the reference book, we optimized the control block using simple 2 and 3 input gates and simple MUX to get the ALUOp signal. In addition to the given table, since we are implementing the full I format instruction set, we needed a little more analysis on truth table, opcode signals to get the required combinational circuit for this block, simplifying the hardware to a great extent.

Since the ALUcontrol and control modules are designed using reduced combinational expressions derived with don't-care conditions, unsupported or illegal instruction encodings are not explicitly handled. For such encodings, the output of the ALU may be undefined. This behavior is intentional and does not constitute a drawback as the design is guaranteed to be exercised only with the specified subset of RISC-V instructions. Exploiting don't-care conditions significantly reduces hardware complexity and gate count, which is appropriate for a constrained instruction subset.

For 64-bit shift-immediate instructions (SLLI, SRLI, SRAI), the shift amount is taken from imm[5:0]

to allow shifts in the range 0–63. The SRLI and SRAI instructions are differentiated using the immediate control bit `imm[10]`, consistent with standard RISC-V encoding conventions for 32 bit numbers.

15 Challenges Faced

One of the primary challenges encountered during the implementation was the integration of multiple functional modules within the top-level wrapper. Managing inter-module connectivity and maintaining consistent and unambiguous naming for internal wires and buses was a tough task.

We also faced difficulties in deriving the combinational logic for the control logic of ALUcontrol and control block and implementing additional I format instructions did make it tougher.